

Medidas de Mitigación y Prevención ante Inyección SQL

1. Descripción del Escenario

Una empresa detectó un intento de **inyección SQL** en el campo de login. La entrada maliciosa fue:

```
' OR '1'='1
```

En respuesta, se decidió implementar medidas inmediatas de **mitigación** y posteriormente aplicar mejoras de **prevención**.

2. Implementación de Medidas de Mitigación

Se desarrolló un **WAF (Web Application Firewall)** en FastAPI, que:

- Bloquea patrones maliciosos (SQLi, XSS, command injection, etc.).
- Cuenta los intentos por IP.
- Bloquea IPs persistentes.
- Registra logs con nivel de detalle (**DEBUG**, **INFO**, **WARNING**).
- Funciona como proxy reverso hacia el backend.
- Permite CORS abierto (solo para laboratorio).

URL del WAF: <http://localhost:8001>

3. Pruebas de Seguridad

Fuerza Bruta / SQL Injection

```
curl -X POST http://localhost:8001/login \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"username":"admin", "password":"pass"}'
```

```
curl -X POST http://localhost:8001/login \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"username":"admin\" OR \"1\"=\"1", "password":"pass"}'
```

```
curl -X POST http://localhost:8001/login \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"username": "\" OR 1=1 --", "password":"any"}'
```

XSS Persistente

```
curl -X POST http://localhost:8001/comment \  
-H "Content-Type: application/json" \  
-d '{"comment": "<script>alert(\"xss\")</script>"}'
```

```
curl -X POST http://localhost:8001/comment \  
-H "Content-Type: application/json" \  
-d '{"comment": "<img src=x onerror=alert(1)>"}'
```

```
curl -X POST http://localhost:8001/comment \  
-H "Content-Type: application/json" \  
-d '{"comment": "<a href=\"javascript:alert(1)\">click</a>"}'
```

Inyección en Headers

```
curl -X POST http://localhost:8001/login \  
-H "User-Agent: <script>alert('x')</script>" \  
-H "Content-Type: application/json" \  
-d '{"username": "test", "password": "test"}'
```

```
curl -X POST http://localhost:8001/login \  
-H "Referer: javascript:alert('xss')" \  
-H "Content-Type: application/json" \  
-d '{"username": "test", "password": "test"}'
```

Bypass WAF con variantes

```
curl -X POST http://localhost:8001/comment \  
-H "Content-Type: application/json" \  
-d '{"comment":"<sCript >alert(String.fromCharCode(88,83,83))</sCript>"}'
```

```
curl -X POST http://localhost:8001/comment \  
-H "Content-Type: application/json" \  
-d '{"comment":"<IMG SRC=\\\"javascript:alert(\\'XSS\\')\\\">"}'
```

```
curl -X POST http://localhost:8001/login \  
-H "Content-Type: application/json" \  
-d '{"username":"admin\\'/**/OR/**/1=1--", "password":"x"}'
```

Simulación de Command Injection

```
curl -X POST http://localhost:8001/execute \  
-H "Content-Type: application/json" \  
-d '{"command":"; whoami"}'
```

4. Clasificación de las Acciones

Acción	Categoría	Justificación
Configurar un WAF	Mitigación	Actúa en tiempo real bloqueando el ataque sin modificar el código
Reescribir el formulario con prepared statements	Prevención	Soluciona la raíz del problema desde el código
Aplicar validación estricta de inputs	Prevención	Restringe entradas inválidas desde el backend
Registrar intentos en logs	Mitigación	No detiene el ataque, pero permite análisis posterior
Capacitar al equipo en OWASP Top 10	Prevención	Reduce errores futuros por desconocimiento

5. Prioridad de Implementación

1. **Reescribir con consultas preparadas** → elimina el vector de ataque.
2. **Configurar el WAF** → protección inmediata mientras se corrige el código.
3. **Registrar intentos en logs** → soporte para análisis forense.
4. **Validación de entradas** → refuerzo del backend.
5. **Capacitación del equipo** → prevención a largo plazo.